Package ‘USFarmSafetyNetLab’

September 11, 2025

Type Package

Title US Farm Safety Net Lab

Version 0.0.0.9000

Author Francis Tsiboe [aut, cre] (<https://orcid.org/0000-0001-5984-1072>)

Maintainer Francis Tsiboe <ftsiboe@hotmail.com>

Contributor -

Reviewer -

Creator Francis Tsiboe

Description This repository centralizes research outputs, analytical tools, and resources for exploring and evaluating the United States agricultural safety net programs. It supports analysis of key programs including the Federal Crop Insurance Program (FCIP), the Noninsured Crop Disaster Assistance Program (NAP), Price Loss Coverage (PLC), Agricultural Risk Coverage (ARC), and various ad-hoc disaster assistance programs.

License GPL-3 + file LICENSE

URL https://github.com/you/USFarmSafetyNetLab

BugReports https://github.com/you/USFarmSafetyNetLab/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

VignetteBuilder knitr

Depends R (>= 4.1.0)

Imports data.table, moments, janitor, readr, stats, stringr, sf, urbnmapr, ggrepel, grid, gridExtra, gtable, ggplot2,
classInt, terra, doBy, plm, sp, spdep, tidyr, tigris, xml2, rfcip

Remotes github::dylan-turner25/rfcip, github::UrbanInstitute/urbnmapr, github::dylan-turner25/rfsa

Suggests dplyr, rvest, purrr, rfcip, knitr, rmarkdown, testthat (>= 3.0.0)

LazyData true

Cite-us If you find it useful, please consider staring the repository and citing the following studies
- Tsiboe, F. and Turner, D. (2025). ``Incorporating buy-up price loss coverage into the United States farm safety net." Applied Economic Perspectives and Policy.
- Tsiboe, F., et al. (2025). ``Risk reduction impacts of crop insurance in the United States."

Applied Economic Perspectives and Policy.
- Gaku, S. and Tsiboe, F. (2024). Evaluation of alternative farm safety net program combination strategies. Agricultural Finance Review.

Contents

analyze_and_summarize_time_series . . . 3
calculate_mode . . . 4
calculate_revenue_moments . . . 5
clean_data . . . 6
clean_eco_share_by_insurance_plan . . . 6
clean_rma_sobtpu . . . 7
clean_sco_share_by_coverage_level . . . 8
clean_supplemental_plan_shares . . . 8
commit_and_push . . . 9
downloaded_nass_large_datasets . . . 10
download_rma_web_data_files . . . 11
enrich_sobtpu_with_covariates . . . 12
enrich_with_commodity_grouping . . . 14
enrich_with_election_grouping . . . 14
enrich_with_location_grouping . . . 15
enrich_with_loss_cause . . . 15
enrich_with_loss_ratio_grouping . . . 16
enrich_with_practice_grouping . . . 17
enrich_with_program_age . . . 17
enrich_with_structural_breaks . . . 18
enrich_with_subsidy_bins . . . 19
ers_theme . . . 19
FCIP_FORCE_AMOUNT_VARIABLES . . . 20
FCIP_FORCE_CHARACTER_KEYS . . . 20
FCIP_FORCE_NUMERIC_KEYS . . . 21
FCIP_INSURANCE_ELECTION . . . 22
FCIP_INSURANCE_ELECTION_RCODED . . . 22
FCIP_INSURANCE_POOL . . . 23
generate_toy_farmer_data . . . 24
get_continuous_rating_parameters . . . 26
get_file_info . . . 27
get_ice_data . . . 28
get_marketing_year_avg_price . . . 29
get_nass_census_data . . . 30
get_nass_historical_track_record_crop . . . 32
get_nass_production_data . . . 33
get_prism_seasonal_data . . . 34
get_state_rental_rates . . . 36
harmonize_codes_and_names . . . 37
harmonize_crop_type_codes . . . 37
layouts_fcip . . . 39
locate_download_link . . . 39
plot_liability_and_acres . . . 40
plot_us_states_choropleth . . . 42
policytime . . . 43
process_nass_dataset . . . 44

push_to_overleaf 45
render_rmarkdown_to_dir 46
rescale_values 47
revealed_aip_state_operation 47
setup_environment 49
setup_git_repo 50
smooth_county_estimates 51
standardize_fcip_column_names 52

Index **53**

`analyze_and_summarize_time_series`
Analyze and summarize time-series data by disaggregate and outcome using data.table

---

Description

This function computes comprehensive summary statistics, trend diagnostics, and both magnitude-sorted and chronological notable changes for each combination of `disaggregate` level and `outcome`.

Usage

```
analyze_and_summarize_time_series(dt, threshold = 10)
```

Arguments

`dt`	A data.table or data.frame containing the columns: • `commodity_year` (integer) • `disaggregate` (character or factor) • `outcome` (character or factor) • `value` (numeric)
`threshold`	Numeric. Percent threshold (e.g., 10 for 10%) used to flag `notable_growth` (>= +threshold) and `notable_decline` (<= -threshold). Defaults to 10.

Value

A named list of outcomes. Each element is a named list of one-row data.frames (one per disaggregate level) containing the following summary columns:

- **mean**: Arithmetic mean of the `value` series.
- **sd**: Standard deviation of the `value` series.
- **median**: Median of the `value` series.
- **iqr**: Interquartile range (75th-25th percentile) of the `value` series.
- **cv**: Coefficient of variation (sd divided by mean).
- **n_obs**: Number of non-missing observations.
- **pct_missing**: Percentage of years with missing `value`.
- **start_year_value**: Value in the first `commodity_year` period.
- **current_year_value**: Value in the last `commodity_year` period.

- **year_with_max_value**: Year when `value` is maximized.
- **max_value**: Maximum `value` observed.
- **year_with_min_value**: Year when `value` is minimized.
- **min_value**: Minimum `value` observed.
- **cumulative_change**: Absolute change (final minus initial value).
- **mean_annual_growth_rate**: Compound annual growth rate (CAGR) in percent.
- **growth_initial_to_final**: Total percent growth from initial to final value.
- **first_diff_sd**: Standard deviation of year-to-year percent changes.
- **max_drawdown_pct**: Greatest percent drop from a peak to subsequent trough.
- **trend_slope**: Slope of linear model `value ~ commodity_year`.
- **trend_p_value**: P-value for slope significance in the trend model.
- **trend_r_squared**: R-squared of the trend model.
- **time_to_peak**: Years from start until the peak year.
- **time_to_trough**: Years from start until the trough year.
- **notable_growth**: Magnitude-sorted list of year: +% changes greater than or equal to threshold.
- **notable_growth_chron**: Chronologically-sorted list of year: +% changes greater than or equal to threshold.
- **notable_decline**: Magnitude-sorted list of year: -% changes less than or equal to -threshold.
- **notable_decline_chron**: Chronologically-sorted list of year: -% changes less than or equal to -threshold.
- **any_missing_years**: Logical flag for missing years in the series.
- **monotonic**: Logical flag if the series strictly increases or decreases.
- **rank**: Rank of the disaggregate within its outcome by descending CAGR.
- **rank_among_outcomes**: Global CAGR rank across all outcome/disaggregate groups.

`calculate_mode` *Calculate the Statistical Mode*

Description

Returns the element that occurs most frequently in a vector.

Usage

```
calculate_mode(x, na.rm = TRUE)
```

Arguments

`x`	A vector of any atomic type (numeric, character, factor,).
`na.rm`	Logical; should missing values be ignored? Defaults to `TRUE`. If `FALSE` and `x` contains any `NA`s, the function returns `NA`.

Details

Internally the function:

1. Optionally removes NAs (`na.rm = TRUE`).
2. Builds a lookup table of unique values via `unique(x)`.
3. Counts the frequency of each unique value with `tabulate(match(x, ux))`.
4. Returns the value with the maximum count.

Because it relies on base R functions, the implementation is vectorised and generally fast for typical data-frame column sizes.

Value

A single value giving the modal element of `x`. If two or more values are tied for the highest frequency, the first one encountered in `x` is returned.

`calculate_revenue_moments` *Calculate key distributional moments for a numeric vector of revenues*

Description

Calculate key distributional moments for a numeric vector of revenues

Usage

```
calculate_revenue_moments(x)
```

Arguments

`x`	Numeric vector of revenues (e.g. per-draw revenues)

Value

A one-row data.frame with:

- mu: mean
- md: median
- sd: standard deviation
- cv: coefficient of variation (sd / mu)
- vr: variance
- sk: skewness
- ku: kurtosis
- lapv: mean squared loss deviations below the mean
- lrpv: lapv / probability of loss
- nlapv: lapv normalized by mu
- nlrpv: lrpv normalized by mu
- lres2: same as lapv (for clarity)
- cdf: empirical probability of loss (P(X < mu))

clean_data *Apply standardized data cleaning opperations*

Description

Apply standardized data cleaning opperations

Usage

```
clean_data(df)
```

Arguments

df	a data frame to clean

Value

a cleaned data frame

Source

copied from https://github.com/dylan-turner25/rmaADM/blob/main/R/helpers.R

Examples

```
## Not run: clean_data(df)
```

clean_eco_share_by_insurance_plan *ECO share by insurance plan*

Description

Computes Enhanced Coverage Option (ECO) **shares of insured acres** for base plans (1=YP, 2=RP, 3=RP-HPE), using ECO plan codes 87-89 mapped back to 1-3, and returns separate shares for ECO-90 and ECO-95.

Usage

```
clean_eco_share_by_insurance_plan(sob)
```

Arguments

sob	A `data.table` (or data.frame) of SOB records that already contain aggregated acre and dollar fields. Must include at least: `insured_acres`, `endorsed_acres`, `insurance_plan_code`, `coverage_level_percent`, `commodity_year`, `state_code`, `county_code`, `commodity_code`, `type_code`, `practice_code`.

Details

- Base data are summed for plans `1:3`.
- ECO records are selected via plan codes `87:89`, remapped to `1:3` (subtract 86), and coverage levels are labeled as `"eco90"` / `"eco95"` (using `coverage_level_percent` * 100).
- ECO shares are computed as `eco_endorsed_acres` / `insured_acres` by key.

Value

A `data.table` with unique rows by `commodity_year, state_code, county_code, commodity_code, type_code, p` and columns: `eco90`, `eco95` (shares in [0,1]); missing combinations may be `NA`.

---

clean_rma_sobtpu — *Clean and aggregate RMA Summary of Business (SOB-TPU) data*

Description

Retrieves RMA SOB-TPU records for requested years, combining **live** years (last 5 years, fetched via `rfcip::get_sob_data()`) with **stable** years (downloaded from a prebuilt `.rds` release), then filters, harmonizes insurance plan codes, coverage levels, and unit structure codes, and returns an analysis-ready `data.table` aggregated to common keys.

Usage

```
clean_rma_sobtpu(
  years = as.numeric(format(Sys.Date(), "%Y")) - 1,
  insurance_plan = NULL,
  acres_only = TRUE,
  addon_only = TRUE,
  harmonize_insurance_plan_code = TRUE,
  harmonize_coverage_level_percent = TRUE,
  harmonize_unit_structure_code = TRUE
)
```

Arguments

`years` Integer vector of commodity years.

`insurance_plan` Optional integer vector of harmonized plan codes to keep after harmonization (1=YP, 2=RP, 3=RP-HPE). If `NULL`, keep all.

`acres_only` Logical; keep only acres-level records. Default `TRUE`.

`addon_only` Logical; exclude CAT (`coverage_type_code == "C"`). Default `TRUE`.

`harmonize_insurance_plan_code` Logical; recode plans to (1,2,3). Default `TRUE`.

`harmonize_coverage_level_percent` Logical; normalize coverage levels to decimal in 0.50 to 0.95 at 0.05 steps. Default `TRUE`.

`harmonize_unit_structure_code` Logical; recode unit structure to (OU, BU, and EU). Default `TRUE`.

Value

A `data.table` aggregated to the keys with columns: `insured_acres`, `endorsed_acres`, `liability_amount`, `total_premium_amount`, `subsidy_amount`, `indemnity_amount`.

---

clean_sco_share_by_coverage_level

SCO share by coverage level

Description

Computes the Supplemental Coverage Option (SCO) **share of insured acres** by coverage level for base plans (1=YP, 2=RP, 3=RP-HPE), using SCO plan codes 31-33 mapped back to 1-3.

Usage

```
clean_sco_share_by_coverage_level(sob)
```

Arguments

`sob`	A `data.table` (or data.frame) of SOB records that already contain aggregated acre and dollar fields. Must include at least: `insured_acres`, `endorsed_acres`, `insurance_plan_code`, `coverage_level_percent`, `commodity_year`, `state_code`, `county_code`, `commodity_code`, `type_code`, `practice_code`.

Details

- Base data are summed for plans `1:3`.
- SCO records are selected via plan codes `31:33` and then remapped to `1:3` (subtract 30) to align with corresponding base plans.
- SCO share is computed as `endorsed_acres / insured_acres` by key and coverage level.

Value

A `data.table` with unique rows by `commodity_year, state_code, county_code, commodity_code, type_code, p` and column: `sco` (share in [0,1]).

---

clean_supplemental_plan_shares

Supplemental plan shares (SCO/ECO)

Description

Aggregates base plan acres/dollars and (optionally) merges in Supplemental Coverage Option (SCO) and Enhanced Coverage Option (ECO) shares for each key.

Usage

```
clean_supplemental_plan_shares(
  sob,
  get_sco_shares = TRUE,
  get_eco_shares = TRUE
)
```

Arguments

sob	A data.table or data.frame of SOB records. Expected columns: insured_acres, endorsed_acres, insurance_plan_code, coverage_level_percent, commodity_year, state_code, county_code, commodity_code, type_code, practice_code, unit_structure_code coverage_type_code, liability_amount, total_premium_amount, subsidy_amount, indemnity_amount.
get_sco_shares	Logical; if TRUE, merge SCO shares by coverage level.
get_eco_shares	Logical; if TRUE, merge ECO-90/95 shares by plan.

Details

Assumes base plans are harmonized to 1=YP, 2=RP, 3=RP-HPE, and coverage_level_percent is in decimals (e.g., 0.90, 0.95). Relies on helpers: clean_sco_share_by_coverage_level() and clean_eco_share_by_insurance_plan().

Value

A data.frame with aggregated acres/dollars and columns sco, eco90, eco95 (shares in [0,1] where available).

---

commit_and_push *Commit and Push Changes to Overleaf Git Repo*

Description

Stages all modified files, commits them with the provided message, and pushes to the Overleaf remote. Automatically handles common push conflicts by attempting a git pull followed by a re-push.

Usage

```
commit_and_push(commit_msg)
```

Arguments

commit_msg	Commit message to use.

Value

Returns TRUE if push succeeded, FALSE otherwise.

downloaded_nass_large_datasets

Download and cache USDA NASS Quick Stats large dataset files

Description

`downloaded_nass_large_datasets()` retrieves a Quick Stats file from the USDA National Agricultural Statistics Service (NASS) https://www.nass.usda.gov/datasets/ page and saves it locally. If the file is already present in the target directory, it is not re-downloaded.

Usage

```
downloaded_nass_large_datasets(
  large_datasets,
  dir_dest = "./data-raw/fastscratch/nass/"
)
```

Arguments

`large_datasets`	`character list` The base name of the Quick Stats file to download. For example, use `"crops"` to fetch `qs.crops_YYYYMMDD.txt.gz` or include `"census2022"` (e.g. `"census2022"`) to fetch the gzipped 2022 census version (`qs.census2022.txt.gz`). any of: "census2002","census2007","census2012","census2017","census2022", "census2007zipcode","census2017zipcode", "animals_products","crops","demographics","economic
`dir_dest`	`character(1)` Path to a directory where downloaded files will be stored. Defaults to `"./data-raw/fastscratch/nass/"`.

Details

1. Prepends `"qs."` to the provided `large_dataset`. If `large_dataset` contains `"census"`, appends `".txt.gz"`, otherwise `NULL`.
2. Ensures `dir_dest` exists (creates it if needed).
3. Scrapes the NASS datasets page (`https://www.nass.usda.gov/datasets/`) for links ending in `.txt.gz`.
4. Downloads the matching file into `dir_dest` if not already present.

Value

Invisibly returns the normalized file large_dataset (e.g. `"qs.crops_YYYYMMDD.txt.gz"` or `"qs.censusYYYY.txt.gz"`) that was downloaded or already present.

Examples

```
## Not run:
# Download the 'crops' dataset if not already cached:
downloaded_nass_large_datasets(large_dataset = "crops")

# Download the 2022 census version:
downloaded_nass_large_datasets(large_dataset = "census2022",
dir_dest = "./data-raw/fastscratch/nass/")

## End(Not run)
```

download_rma_web_data_files

Download and Process RMA Web Data Files

Description

Fetches one or more years of USDA RMA Summary of Business data (state-county-crop & livestock participation series), unzips, reads the pipe-delimited text, applies the correct column names, and saves each year as an `.rds` under `dest`.

Usage

```
download_rma_web_data_files(
  years,
  file_name,
  dest = "./data-raw/data_release",
  url_rma_ftp_file_access = "https://pubfs-rma.fpac.usda.gov/pub"
)
```

Arguments

`years`	Integer vector of crop or livestock years to fetch.
`file_name`	Character; one of `"sobcov"`, `"sobtpu"`, `"lgm"`, `"lrp"`, or `"colsom"`. Determines both the subdirectory and the column schema.
`dest`	Character path where the final `.rds` files will go. Defaults to `"./data-raw/data_release"`.
`url_rma_ftp_file_access`	Base URL for USDA RMA web data. Defaults to the official RMA FTP file access root.

Value

Invisibly returns `NULL`. Side effect: one `.rds` per year is written under `dest`, named `<file_name>_<year>.rds`.

Examples

```
## Not run:
# Get sobtpu data for 2018-2022
download_rma_web_data_files(2018:2022, "sobtpu")

## End(Not run)
```

enrich_sobtpu_with_covariates

Enrich FCIP table (modular)

Description

Convenience wrapper around `enrich_sobtpu()` that exposes the same arguments and returns the enriched table. This function exists for user-facing discoverability while deferring all work to `enrich_sobtpu()`.

Usage

```
enrich_sobtpu_with_covariates(
  sob,
  election_grouping = TRUE,
  commodity_grouping = TRUE,
  practice_grouping = TRUE,
  loss_ratio_grouping = TRUE,
  program_age_grouping = TRUE,
  loss_cause_grouping = TRUE,
  location_grouping = TRUE,
  structural_breaks = TRUE,
  subsidy_bins = TRUE,
  practice_map = NULL,
  commodity_map = NULL,
  ers_xls = NULL,
  crd_csv = NULL,
  base_year_loss = 1999L,
  sobcov = NULL,
  sobscc = NULL,
  colsom = NULL
)
```

Arguments

`sob`
: A `data.frame` or `data.table` with FCIP-like columns. At minimum, the following columns are required by the modules you enable:
  - **Election**: `insurance_plan_code`, `coverage_level_percent`, `unit_structure_code`
  - **Practice**: `commodity_year`, `commodity_code`, `practice_code`
  - **Commodity**: `commodity_year`, `commodity_code`, `insured_acres`
  - **Loss ratio**: `state_code`, `county_code`, `CROP`, `commodity_year`, `indemnity_amount`, `total_premium_amount`
  - **Program age**: `state_code`, `county_code`, `commodity_code`, `commodity_year`
  - **Loss cause**: `state_code`, `county_code`, `commodity_code`, `commodity_year`
  - **Location**: `state_code`, `county_code`
  - **Structural breaks**: `commodity_year`
  - **Subsidy bins**: `subsidy_amount`, `total_premium_amount`

`election_grouping`
: Logical. Add plan/product, coverage buckets, and unit structure columns (`PLAN`, `RPYP`, `COV`, `STRUCT`). Default `TRUE`.

`commodity_grouping`
: Logical. Join commodity groupings, normalize `CROP`, and add `TOPCROP` (top 12 by insured acres). Default `TRUE`.

`practice_grouping`
: Logical. Join irrigation/organic practice recodes (`irrigation_recode`, `organic_recode`). Default `TRUE`.

`loss_ratio_grouping`
: Logical. Compute cumulative prior-years loss ratio (`cum_lr`) and bins (`CLRCAT`, `CLRmCAT`) by state-county-crop-year. Default `TRUE`.

`program_age_grouping`
: Logical. Compute program age (span of prior positive-liability years) and 10/5/4-year bins (`program_age_by_year`, `program_age_by_year_10`, `program_age_by_year_05`, `program_age_by_year_04`). Default `TRUE`.

`loss_cause_grouping`
: Logical. Compute cause-of-loss shares for area and indemnity (`evntA_*`, `evntI_*`) and binary flags (`CAT_evntA_*`, `CAT_evntI_*`). Default `TRUE`.

`location_grouping`
: Logical. Add state name/abbreviation, ERS region, and NASS CRD (`COUNTY`, `ERSReg`, `ERSReg_cd`, `CRD`). Default `TRUE`.

`structural_breaks`
: Logical. Add farm-bill era labels (`period_farmbill`) and a pre/post 2012 indicator (`period_combo`). Default `TRUE`.

`subsidy_bins`
: Logical. Add coarse subsidy-share bins (e.g., `SUB040`, `SUB042`, ..., `SUB080`). Default `TRUE`.

`practice_map`
: Optional `data.frame`/`data.table` for practice recodes. If `NULL`, it is downloaded. Must include: `commodity_year`, `commodity_code`, `practice_code`, `irrigation_recode`, `organic_recode`.

`commodity_map`
: Optional `data.frame`/`data.table` for commodity groupings. If `NULL`, it is downloaded. Must include: `commodity_year`, `commodity_code`, `commodity_name`, `CROP`, `commodity_group`.

`ers_xls`
: Optional file path to the ERS region Excel file. If `NULL`, it is downloaded.

`crd_csv`
: Optional file path to the NASS CRD CSV. If `NULL`, it is downloaded.

`base_year_loss`
: Integer. Base year for the loss-ratio cumulative window (prior-year sums use years >= `base_year_loss` and < `current year`). Default `1999L`.

`sobcov`
: Optional preloaded SOBCOV aggregate used by the program-age and loss-cause modules. If `NULL`, it is downloaded. Must include: `commodity_year`, `commodity_code`, `state_code`, `county_code`, `net_reported_quantity` (or `net_acre_qty`), and `liability_amount` (or `liability_amt`). If `net_acre_typ` exists, rows containing "ACRE" are retained.

`sobscc`
: Optional preloaded SOBSCC aggregate used by the program-age module. If `NULL`, it is downloaded. Same column requirements as `sobcov`.

`colsom`
: Optional preloaded Cause-of-Loss detail used by the loss-cause module. If `NULL`, it is downloaded. Must include: `commodity_year` (or `crop_yr`), `commodity_code` (or `crop_cd`), `state_code` (or `state_cd`), `county_code` (or `county_cd`), `damage`, `det_acre_qty`, `indem_amt`.

Value

A `data.table` equal to `sob` with the selected modules added via left joins. If a `liability_amount` column exists, rows with non-finite or non-positive values are filtered at the end.

enrich_with_commodity_grouping

Add commodity groupings and TOPCROP tag (left join)

Description

Add commodity groupings and TOPCROP tag (left join)

Usage

```
enrich_with_commodity_grouping(sob, commodity_map = NULL, top_n = 12L)
```

Arguments

sob	data.frame/data.table with commodity_year, commodity_code, insured_acres
commodity_map	Optional mapping with columns commodity_year, commodity_code, commodity_name, CROP, commodity_group. If NULL, it will be downloaded.
top_n	integer. Number of crops to keep as named in TOPCROP; others -> "OTHER".

Value

data.table with commodity_name, CROP, commodity_group, TOPCROP

enrich_with_election_grouping

Add plan, product, coverage buckets, and unit structure labels

Description

Add plan, product, coverage buckets, and unit structure labels

Usage

```
enrich_with_election_grouping(sob)
```

Arguments

sob — data.frame/data.table with columns:

- insurance_plan_code
- coverage_level_percent (either 0-1 or 0-100)
- unit_structure_code

Value

data.table with new columns: PLAN, RPYP, COV, STRUCT

enrich_with_location_grouping

Add state names/abbreviations, ERS Region, and NASS CRD (left joins)

Description

Add state names/abbreviations, ERS Region, and NASS CRD (left joins)

Usage

```
enrich_with_location_grouping(sob, ers_xls = NULL, crd_csv = NULL)
```

Arguments

sob	data.frame/data.table with state_code, county_code
ers_xls	Optional path to the ERS excel file; if NULL, it is downloaded.
crd_csv	Optional path to the CRD csv file; if NULL, it is downloaded.

Value

data.table with state_abbreviation, state_name, ERSReg, ERSReg_cd, CRD

enrich_with_loss_cause

Add cause-of-loss shares (area & indemnity) and binary flags

Description

Downloads/uses Cause-of-Loss (COL) detail and SOBCOV aggregates to compute, for each {state, county, commodity, year}, the percent of area and percent of indemnity attributable to broad event categories (drought, heat, cold, moisture, biotic, other). Results are joined back to sob as:

- evntA_*: percent of area (0-100) by event
- evntI_*: percent of indemnity (0-100) by event
- CAT_evntA_*, CAT_evntI_*: binary indicators (>0 to 1)

Usage

```
enrich_with_loss_cause(sob, colsom = NULL, sobcov = NULL)
```

Arguments

sob	data.frame/data.table with keys: state_code, county_code, commodity_code, commodity_year.
colsom	Optional COL detail table. If NULL, downloads col/colsom_all.rds. Must have: (crop_yr|commodity_year), (crop_cd|commodity_code), (state_cd|state_code), (county_cd|county_code), damage, net_determined_quantity, indemnity_amount.
sobcov	Optional SOBCOV table for denominators. If NULL, downloads sob/sobcov_all.rds. Must have: (crop_yr|commodity_year), (crop_cd|commodity_code), (state_cd|state_code (county_cd|county_code), net_reported_quantity|net_acre_qty, liability_amount|lial and optionally net_acre_typ (rows containing "ACRE" are kept if present).

Details

Event recodes:

- **evnt*_drght**: DROUGHT; FAILURE OF IRRIGATION SUPPLY/EQUIPMENT; INABILITY TO PREPARE LAND FOR IRRIGATION; DROUGHT DEVIATION
- **evnt*_tempH**: EXCESS SUN; HEAT; HOT WIND
- **evnt*_tempL**: COLD WET WEATHER; COLD WINTER; FREEZE; FROST
- **evnt*_moist**: EXCESS MOISTURE/PRECIPITATION/RAIN; FLOOD; POOR DRAINAGE; EROSION
- **evnt*_biotc**: PLANT DISEASE; WILDLIFE; INSECTS; MYCOTOXIN; ASIAN SOYBEAN RUST; ASIATIC CITRUS CANKER; DISEASE, AQUACULTURE; MEDFLY; MYCOTOXIN (AFLATOXIN)
- **evnt*_other**: all other/uncoded damages

Shares are capped to 100 and missing shares are filled with 0.

Value

data.table = sob with new evntA_*, evntI_*, CAT_evntA_*, CAT_evntI_* columns.

---

enrich_with_loss_ratio_grouping *Add cumulative loss ratio (prior years) and LR bins*

Description

Computes cumulative indemnity and premium over *prior* years (from base_year) for each (state, county, crop), derives:

- cum_lr: cumulative loss ratio up to previous year
- CLRCAT: bins (less than or equal to 0.50, 0.50-0.65, ..., greater than 5.00)
- CLRmCAT: bins (less than or equal to 0.50, 0.50-0.65, ..., greater than 2.20) and LEFT-joins these onto sob.

Usage

```
enrich_with_loss_ratio_grouping(sob, base_year = 1999L)
```

Arguments

sob	data.frame/data.table with: state_code, county_code, CROP, commodity_year, indemnity_amount, total_premium_amount
base_year	integer. First year included in the cumulative window. Default 1999L.

Value

data.table (enriched sob)

---

enrich_with_practice_grouping

Add irrigation/organic practice recodes (left join)

Description

Add irrigation/organic practice recodes (left join)

Usage

```
enrich_with_practice_grouping(sob, practice_map = NULL)
```

Arguments

sob	data.frame/data.table with `commodity_year`, `commodity_code`, `practice_code`
practice_map	Optional data.frame/data.table with columns `commodity_year`, `commodity_code`, `practice_code`, `irrigation_recode`, `organic_recode`. If `NULL`, it will be downloaded from the project release.

Value

data.table with `irrigation_recode`, `organic_recode`

---

enrich_with_program_age

Attach "program age" by (state, county, commodity, year)

Description

Computes, for each row's (state_code, county_code, commodity_code, commodity_year), the **span of years observed prior to the current year** with positive liability in the combined SOBCOV/SOBSCC aggregates:

Usage

```
enrich_with_program_age(sob, sobcov = NULL, sobscc = NULL)
```

Arguments

sob	data.frame/data.table with (required) columns: state_code, county_code, commodity_code, commodity_year.
sobcov	Optional data.frame/data.table for SOBCOV; if NULL, it is downloaded. Must contain commodity_year, commodity_code, state_code, county_code, net_reported_quanti liability_amount.
sobscc	Optional data.frame/data.table for SOBSCC; if NULL, it is downloaded. Same column expectations as sobcov.

Details

$$program_age_by_year(y) = \max\{t < y\} - \min\{t < y\}, \quad t \in \text{years with liab_actual} > 0$$

If there is no prior year with positive liability, the age is set to 0. Binned variants at widths 10/5/4 years are also added.

- Uses efficient cumulative min/max over a union timeline of **target years** (from sob) and **observed positive-liability years** (from SOBCOV/SOBSCC) per group.
- All joins are left joins; no rows are dropped from sob.
- Age is computed on the year scale (integers) and **does not add 1** to the span, matching your original specification ($\max - \min$).

Value

data.table equal to sob with new columns: program_age_by_year, program_age_by_year_10, program_age_by_year_05, program_age_by_year_04.

---

enrich_with_structural_breaks

Add farm-bill era labels and a before/after 2012 combo

Description

Add farm-bill era labels and a before/after 2012 combo

Usage

```
enrich_with_structural_breaks(sob)
```

Arguments

sob	data.frame/data.table with commodity_year

Value

data.table with period_farmbill, period_combo

enrich_with_subsidy_bins

Add coarse subsidy bins (e.g., SUB040, SUB042, ..., SUB080)

Description

Add coarse subsidy bins (e.g., SUB040, SUB042, ..., SUB080)

Usage

```
enrich_with_subsidy_bins(sob, lo = 0.4, hi = 0.8, step = 0.02)
```

Arguments

`sob`	data.frame/data.table with `subsidy_amount`, `total_premium_amount`
`lo`, `hi`	numeric lower/upper clamp for ratio; defaults 0.40-0.80
`step`	numeric bin width; default 0.02

Value

data.table with `subsidy_bins` (character labels)

ers_theme

ERS Theme

Description

ERS Theme

Usage

```
ers_theme()
```

Source

coppied from https://github.com/USDA-REE-ERS/MTED-Theme on 08/01/2025

Examples

```
ggplot2::ggplot() + ers_theme()
```

FCIP_FORCE_AMOUNT_VARIABLES

Column names to coerce to numeric

Description

A character vector of column names that should be converted to character during data ingestion and cleaning.

Usage

```
FCIP_FORCE_AMOUNT_VARIABLES
```

Format

A `character` vector of column names.

Value

A `character` vector of field names to coerce to numeric

FCIP_FORCE_CHARACTER_KEYS

Column names to coerce to character

Description

A character vector of column names that should be converted to character during data ingestion and cleaning.

Usage

```
FCIP_FORCE_CHARACTER_KEYS
```

Format

A `character` vector of column names.

Value

A `character` vector of field names to coerce to character

FCIP_FORCE_NUMERIC_KEYS

Column names to coerce to numeric

Description

A character vector of column names that should be converted from character to numeric during data ingestion and cleaning.

Usage

```
FCIP_FORCE_NUMERIC_KEYS
```

Format

A `character` vector of column names.

Details

The following fields, although often stored as text, represent numeric values and must be coerced for accurate calculation and analysis:

- `commodity_year`: Crop year of the record.
- Fields in `FCIP_INSURANCE_POOL`: `state_code`, `county_code`, `commodity_code`, `type_code`, `practice_code`.
- `record_category_code`: Category of the record.
- `insurance_plan_code`: Code for the insurance plan.
- `coverage_level_percent`: Coverage level percentage.

Value

A `character` vector of field names to coerce to numeric.

Examples

```
## Not run:
# View default keys
FCIP_FORCE_NUMERIC_KEYS

# Extend with a custom numeric field
rFarmPolicySim:::FCIP_FORCE_NUMERIC_KEYS <- c(
  rFarmPolicySim:::FCIP_FORCE_NUMERIC_KEYS,
  "custom_numeric_field"
)

## End(Not run)
```

FCIP_INSURANCE_ELECTION

Insurance election identifier fields

Description

A character vector of column names that define an insurance election within the Federal Crop Insurance Program (FCIP). Each field corresponds to an attribute of a policy election.

Usage

```
FCIP_INSURANCE_ELECTION
```

Format

A `character` vector of field names:

unit_structure_code Structure of the insured unit (e.g., basic, optional).

insurance_plan_code Code for the insurance product (e.g., MPCI, CRC).

coverage_type_code Type of coverage (e.g., Actual/Assumed Yield, Yield Protection).

coverage_level_percent Elected coverage level as a percentage of approved yield or price.

Value

A `character` vector of field names used to specify insurance elections.

Examples

```
## Not run:
# Default election fields
FCIP_INSURANCE_ELECTION

# Override to include only plan and coverage level
rFarmPolicySim:::FCIP_INSURANCE_ELECTION <- c(
  "insurance_plan_code",
  "coverage_level_percent"
)

## End(Not run)
```

FCIP_INSURANCE_ELECTION_RCODED

Insurance election identifier fields (recoded)

Description

A character vector of recoded column names that specify an insurance election within the Federal Crop Insurance Program (FCIP). These fields correspond to recoded versions of the original election attributes.

Usage

```
FCIP_INSURANCE_ELECTION_RCODED
```

Format

A `character` vector of field names:

unit_structure_recode Recoded unit structure (e.g., basic, optional).

insurance_plan_recode Recoded insurance plan code (e.g., APH, YP).

coverage_type_code Coverage type code (unchanged).

coverage_level_percent unchanged

Value

A `character` vector of recoded insurance election field names.

Examples

```
## Not run:
# View the default recoded election fields
FCIP_INSURANCE_ELECTION_RCODED

# Override to drop the recoded plan field
rFarmPolicySim::FCIP_INSURANCE_ELECTION_RCODED <- c(
  "unit_structure_recode",
  "coverage_type_code",
  "coverage_level_percent"
)

## End(Not run)
```

FCIP_INSURANCE_POOL *Insurance pool identifier fields*

Description

A character vector of column names that together define a unique insurance pool in the Federal Crop Insurance Program (FCIP).

Usage

```
FCIP_INSURANCE_POOL
```

Format

A `character` vector of field names.

Details

Insurance pools represent the most granular level of rate making within FCIP. Each pool is uniquely identified by the combination of:

- **state_code**: State FIPS code
- **county_code**: County FIPS code
- **commodity_code**: Crop commodity code
- **type_code**: Crop type (e.g., grain vs. silage)
- **practice_code**: Production practice (e.g., irrigated, organic)

Value

A `character` vector specifying the columns used to define each FCIP insurance pool.

Examples

```
## Not run:
# Default insurance pool fields
FCIP_INSURANCE_POOL

# Override to a subset of the original fields
rFarmPolicySim:::FCIP_INSURANCE_POOL <- c(
  "state_code", "county_code", "commodity_code"
)

## End(Not run)
```

---

generate_toy_farmer_data

Generate a sample farmer-level toy dataset for testing

Description

Generate a sample farmer-level toy dataset for testing

Usage

```
generate_toy_farmer_data(
  n = 100,
  year = 2020,
  state = 18,
  counties = 7:23,
  commodity = 41,
  types = c(26, 16),
  practices = c(2, 3, 997),
  unit_codes = c("BU", "OU", "EU"),
  plan_codes = c(1:6, 16:17, 90),
  coverage_codes = c("A", "C"),
  cov_level_range = c(0.5, 0.95),
  yield_mean = 100,
  yield_sd = 10,
```

```
  price_mean = 2.5,
  price_sd = 1.5,
  price_share = NULL,
  area_share = NULL,
  planted_area = NULL,
  production_history_length = 0:10
)
```

Arguments

`n`	Integer number of farmer records to generate. Default 100.
`year`	Numeric value of commodity year. Default 2020.
`state`	Numeric vector of state FIPS codes. Default 19.
`counties`	Numeric vector of county FIPS codes. Default 127.
`commodity`	Numeric vector of commodity codes. Default 41.
`types`	Numeric vector of type codes. Default 16.
`practices`	Numeric vector of practice codes. Default c(2, 3, 997).
`unit_codes`	Character vector of unit structure codes. Default c("BU", "OU", "EU").
`plan_codes`	Numeric vector of insurance plan codes. Default c(1:6, 16:17, 90).
`coverage_codes`	Character vector of coverage type codes. Default c("A", "C").
`cov_level_range`	Numeric vector of length 2 giving the min and max for coverage_level_percent. Default c(0.8, 0.8).
`yield_mean`	Numeric mean for approved_yield and rate_yield. Default 100.
`yield_sd`	Numeric standard deviation for approved_yield and rate_yield. Default 10.
`price_mean`	Numeric mean for expected price. Default 2.5.
`price_sd`	Numeric standard deviation for expected price. Default 1.5.
`price_share`	Numeric
`area_share`	Numeric
`planted_area`	Numeric
`production_history_length`	Integer vector of indices for actual_farm_yield columns. Default 0:10.

Value

A `data.table` with one row per farmer and columns:

- `farmid`
- `commodity_year`
- `state_code`, `county_code`
- `commodity_code`, `type_code`, `practice_code`
- `unit_structure_code`, `insurance_plan_code`
- `coverage_type_code`, `coverage_level_percent`
- `approved_yield`, `rate_yield`
- `reported_acres`, `insured_share_percent`
- `price_election_percent`, `dmage_area_rate`, `market_year_price`
- `actual_farm_yield_0` to `actual_farm_yield_<max(production_history_length)>`

Examples

```
farmdata <- generate_toy_farmer_data()
farmdata <- generate_toy_farmer_data(
  n = 20,
  cov_level_range = c(0.7, 0.9),
  production_history_length = 0:2
)
head(farmdata)
```

---

get_continuous_rating_parameters

Compute continuous rating parameters from ADM data for a given year

---

Description

Pulls ADM datasets A01010_BaseRate (Base Rate) and A01040_CoverageLevelDifferential (Coverage Level Differential), harmonizes plan codes (90 to 1, 44 to 2, 25/42 to 3), aggregates base quantities, fits a commodity-level regression to estimate missing coverage-level differentials, and returns per-(state, county, commodity, type, practice, plan) records with:

- Ycr (reference amount), Expo (exponent), Rr (fixed rate), Rf (reference rate)
- PF (ratio Rr / max Rr within county/commodity/type/plan)
- Rd50..Rd85 (coverage differential factors; Rd65 anchored at 1)
- Fu_EU / Fu_BU (enterprise/basic unit residual factors)

Usage

```
get_continuous_rating_parameters(year)
```

Arguments

`year`	integer scalar (e.g., 2023)

Details

Coverage-level differentials (Rd*) are modeled, for each commodity (crop), as a multivariate quadratic in coverage level (and its square) and the base fixed rate Rr (and its square), including an interaction with coverage level. The 65% level is excluded from the regression fit and then anchored to 1 (i.e., Rd65 = 1) when filling. Coverage levels considered are 50-85%.

Rd* values are averaged across insurance plans at the county-commodity level before joining back to the base table; if plan-specific differentials are required, retain `insurance_plan_code` in the differential aggregation.

Value

A `data.table` with columns:

- `state_code`, `county_code`, `commodity_code`, `type_code`, `practice_code`, `insurance_plan_code`
- `Ycr`, `Expo`, `Rr`, `Rf`, `PF`
- `Rd50`, `Rd55`, `Rd60`, `Rd65`, `Rd70`, `Rd75`, `Rd80`, `Rd85`
- `Fu_EU`, `Fu_BU`
- `commodity_year`

Note

PF is computed as `Rr / max(Rr)` within (state, county, commodity, type, insurance_plan_code).

For observations with missing Rd*, estimated values follow the approach described by Coble et al. (2010), specifying a quadratic function of coverage level and the underlying rate terms, with the 65% level as the normalization point.

References

Coble, K. H., Knight, T. O., Goodwin, B. K., Miller, M. F., Rejesus, R. M., & Duffield, G. (2010). *A Comprehensive Review of the RMA APH and COMBO Rating Methodology: Final Report*. USDA Risk Management Agency. https://www.rma.usda.gov/sites/default/files/topics/comprehensivereview.pdf

---

`get_file_info` *Get File Information from a Directory*

Description

Scans a specified directory for files with a given suffix and returns a data frame containing their file paths, sizes in bytes, and sizes in megabytes.

Usage

```
get_file_info(directory = "./data-raw", file_suffix = ".rds")
```

Arguments

`directory`	A character string specifying the path to the directory to scan. Defaults to `"./data-raw"`.
`file_suffix`	A character string specifying the file suffix to match. Defaults to `".rds"`.

Value

A data frame with columns:

`file_path`	Full file path
`size_bytes`	File size in bytes
`size_mb`	File size in megabytes

Source

copied from https://github.com/dylan-turner25/rmaADM/blob/main/R/helpers.R

Examples

```
## Not run:
get_file_info()
get_file_info(directory = "./my-data", file_suffix = ".csv")

## End(Not run)
```

---

get_ice_data — *Download and clean Insurance Control Elements tables*

Description

`get_ice_data()` retrieves all "YTD" ICE (Insurance Control Elements) text files from the specified directory on the RMA public FTP site for one or more years, downloads them to a temporary location, reads them as pipe-delimited data, applies internal cleaning routines, and returns the combined dataset. Original text files are discarded after reading.

Usage

```
get_ice_data(
  years = 2012,
  ice_url =
   "https://pubfs-rma.fpac.usda.gov/pub/References/insurance_control_elements/PASS/",
  selected_ice = NULL
)
```

Arguments

`years`	Integer vector of calendar years to download (e.g. `2012:2020`). Defaults to `2012`.
`ice_url`	Character string giving the base URL of the ICE directory on the RMA FTP site. Must end with a slash. Defaults to `"https://pubfs-rma.fpac.usda.gov/pub/References/insur`
`selected_ice`	Character vector of keyword(s) or regular expressions to filter the filenames. Only ICE files whose names match at least one element of `selected_ice` will be downloaded. If `NULL`, all "YTD" files are processed.

Value

A single `data.table` (invisibly coercible to `data.frame`) containing the cleaned ICE data for all requested years. If no matching files are found or all downloads fail, returns an empty `data.table`.

Examples

```
## Not run:
# Download & process ICE tables for 2018 and 2019,
# filtering for any file with "IceAOExpenseSubsidy" in its name
ice_df <- get_ice_data(
  years        = 2018:2019,
  selected_ice = "IceAOExpenseSubsidy"
)

## End(Not run)
```

---

get_marketing_year_avg_price

Get Marketing Year Average Price for a Single Crop from USDA NASS Quick Stats

---

Description

`get_marketing_year_avg_price()` fetches USDA NASS Quick Stats data for a specified crop (`short_desc`) at one or more aggregation levels (`agg_level_desc`), computes the mean price for the marketing year, joins to official RMA commodity codes, applies necessary unit conversions, and returns a tidy table of marketing-year average prices.

Usage

```
get_marketing_year_avg_price(
  dir_source = "./data-raw/fastscratch/nass/",
  agg_level_desc = c("NATIONAL", "STATE", "COUNTY"),
  short_desc = "CORN, GRAIN - PRICE RECEIVED, MEASURED IN $ / BU"
)
```

Arguments

`dir_source`	`character(1)` Path to the directory where Quick Stats files are stored. Defaults to `"./data-raw/fastscratch/nass/"`.
`agg_level_desc`	`character` One or more values for the `agg_level_desc` field in the Quick Stats data. Can be `"STATE"` or/and `"NATIONAL"`. Defaults to `"NATIONAL"`.
`short_desc`	`character` One or more Quick Stats `short_desc` strings identifying the crop-price series to retrieve. Defaults to `"CORN, GRAIN - PRICE RECEIVED, MEASURED IN $ / BU"`. **Currently, only the following set is supported:** c("OATS - PRICE RECEIVED, MEASURED IN $ / BU", "RYE - PRICE RECEIVED, MEASURED IN $ / BU", "TOBACCO - PRICE RECEIVED, MEASURED IN $ / LB", "CORN, GRAIN - PRICE RECEIVED, MEASURED IN $ / BU", "FLAXSEED - PRICE RECEIVED, MEASURED IN $ / BU", "BARLEY - PRICE RECEIVED, MEASURED IN $ / BU", "BEANS, DRY EDIBLE, INCL CHICKPEAS - PRICE RECEIVED, MEASURED IN $ / CWT", "HAY - PRICE RECEIVED, MEASURED IN $ / TON", "WHEAT - PRICE RECEIVED, MEASURED IN $ / BU", "COTTON - PRICE RECEIVED, MEASURED IN $ / LB", "SORGHUM, GRAIN - PRICE RECEIVED, MEASURED IN $ / CWT", "SOYBEANS -

PRICE RECEIVED, MEASURED IN $ / BU", "SUGARBEETS - PRICE RECEIVED, MEASURED IN $ / TON", "PEAS, DRY EDIBLE - PRICE RECEIVED, MEASURED IN $ / CWT", "SUNFLOWER - PRICE RECEIVED, MEASURED IN $ / CWT", "RICE - PRICE RECEIVED, MEASURED IN $ / CWT", "PEANUTS - PRICE RECEIVED, MEASURED IN $ / LB", "CANOLA - PRICE RECEIVED, MEASURED IN $ / CWT", "MAPLE SYRUP - PRICE RECEIVED, MEASURED IN $ / GALLON", "RICE, LONG GRAIN - PRICE RECEIVED, MEASURED IN $ / CWT", "MILLET, PROSO - PRICE RECEIVED, MEASURED IN $ / BU", "SUGARCANE - PRICE RECEIVED, MEASURED IN $ / TON", "SAFFLOWER - PRICE RECEIVED, MEASURED IN $ / CWT")

Value

A `data.table` with columns:

- `commodity_year` (`integer`): the marketing year
- `commodity_code` (`integer`): NASS commodity code
- `state_code` (`integer`): state FIPS code (if `agg_level_desc` includes `"STATE"`)
- `marketing_year_avg_price` (`numeric`): average price for the marketing year, in dollars per unit
- `data_source` (`character`): always `"USDA NASS Quick Stats"`

See Also

- `process_nass_dataset()` for the underlying data fetch and filtering
- `get_nass_large_datasets()` for downloading the raw Quick Stats files

Examples

```
## Not run:
# Default: national average for corn
get_marketing_year_avg_price()

# Both state and national for wheat
get_marketing_year_avg_price(
  agg_level_desc = c("STATE", "NATIONAL"),
  short_desc     = "WHEAT - PRICE RECEIVED, MEASURED IN $ / BU"
)

## End(Not run)
```

`get_nass_census_data` *Prepare USDA NASS Census Data for Release*

Description

Iterates over one or more USDA NASS census years, fetching each via `process_nass_dataset()`, and produces three sets of state-level and county-level summaries for each year:

1. Agricultural land metrics by state,
2. Crop insurance totals by state and county,
3. Broad Farm Registry (BRF) census summaries by state and national level.

Usage

```
get_nass_census_data(
  censuses = c(2022, 2017, 2012, 2007, 2002),
  dir_source = "./data-raw/fastscratch/nass/",
  dir_dest = "data-raw/data_release/nass/"
)
```

Arguments

`censuses`	Integer vector. One or more census years to process (e.g. `c(2022, 2017, 2012, 2007, 2002)`).
`dir_source`	Character. Path to the directory containing raw NASS Quick Stats census datasets (default: `"./data-raw/fastscratch/nass/"`).
`dir_dest`	Character. Directory where the processed RDS files will be saved (default: `"data-raw/data_release/nass/"`).

Details

For each year in `censuses`, the function: First, it calls `process_nass_dataset()` with the `large_dataset = paste0("census", census)` argument to load the raw census data into a `data.table`, then renames `commodity_year` to `census_year`. It then performs three blocks of aggregation:

1. **Agricultural Land by State** Filters for ECONOMICS-sector state-level records on cropland, pasture, woodland, and cropland share, cleans and converts the `value` field to numeric, computes the mean by `(census_year, state_code, short_desc)`, recodes `short_desc` to simple labels (`cropland`, `pasture`, `woodland`, `cropland_pct`), pivots wide with `data.table::dcast()`, coerces `census_year` and `state_code` to numeric, and saves `nass_census_agLand_state_<census>.rds`.
2. **Crop Insurance Summaries** Subsets for both cropland and crop-insurance acreage and operation counts plus several farm-related income receipt categories, strips commas and coerces `value` to numeric, drops invalid entries, converts location codes to numeric, filters to insurance-related domains, sums `value` by all relevant grouping fields, and saves `nass_census_insurance_data_<`
3. **BRF Census Aggregates** Filters for producer counts and acres (owned vs. rented), converts `value` to numeric, removes zeros and missing, first computes group-wise means and then sums across `(census_year, state_code, state_alpha, agg_level_desc, unit_desc, domaincat_desc)`, recodes `domaincat_desc` to `ALL_` or `BRF_` plus unit, pivots wide, reorders and renames columns, and saves `nass_census_brf_<census>.rds`.

Value

Character vector of all `nass_census_*.rds` filenames written to `dir_dest`.

See Also

- `process_nass_dataset` for loading raw Quick Stats data

get_nass_historical_track_record_crop

Download and Process USDA NASS Historical Crop Track Records

Description

Scrapes the USDA NASS historical track records web page to identify and download the latest `croptrXX.zip` archive, parses its index to locate CSV tables for area planted and harvested by crop, reads and reshapes each table, cleans and standardizes units and variable names, applies special-case unit conversions, and returns a unified time series data frame of crop-level measures (area planted, area harvested, production, yield, price).

Usage

```
get_nass_historical_track_record_crop(
  url = "https://usda.library.cornell.edu/concern/publications/c534fn92g?locale=en",
   dir_source = "./data-raw/fastscratch/nass/"
)
```

Arguments

`url`	Character. URL of the USDA NASS historical track record page.
`dir_source`	Character. Directory into which to download and extract the ZIP file. Default is `"./data-raw/fastscratch/nass/"`. Must exist or be creatable.

Details

1. **Link discovery & download**
   - Reads all `<a>` hrefs from `url`, filters for links containing `"c534fn92g"`, `"zip"`, and `"croptr"`.
   - Extracts the year from each filename and selects the link with the maximum year.
   - Downloads that single ZIP as `croptr.zip` into `dir_source`.
2. **Index parsing**
   - Opens `crop_index.htm` inside the ZIP, extracts the second HTML table as text.
   - Splits on double line breaks, filters to lines mentioning "Area Planted" or "Area Harvested", and excludes summary or non-crop entries.
   - Splits each line into `Page`, `File`, `Description`, then further into `Description` and a year range (`start`/`end`), and derives the uppercase crop name.
3. **CSV reading & reshaping**
   - For each row of the index:
     - Reads the CSV inside the ZIP once to locate header (`h`), unit (`u`), and data (`d`) rows.
     - Reads it again skipping to the data start, then loops over each data column to build a long table with columns `crop`, `crop_yr`, `variable`, `unit`, and `value`.
   - Combines all tables into one data.frame.
4. **Cleaning & standardization**
   - Converts `crop_yr` and `value` to numeric, drops zeros and invalids.
   - Scales values when unit indicates thousands, strips formatting characters, and normalizes variable names.

- Consolidates crop subtypes (e.g. grain/silage/grazed) under a single `crop`.

5. **Variable mapping & special conversions**
   - Identifies key measures via lookup lists (`Area Planted`, `Area Harvested`, `Production`, `Yield`, `Price`) and relabels them to `"Tracks_*"`.
   - Applies special-case unit conversions (cents to dollars, bales to lbs, cwt to lbs, tons to lbs, etc.), including crop-specific rules (cotton, canola, dry beans, rice, sugarbeet, sugarcane, sunflower).
6. **Final pivot**
   - Pivots the cleaned long table to wide format so each `"Tracks_*"` measure is its own column, and returns a data.frame with columns: `CROP`, `crop_year`, `Tracks_area_planted`, `Tracks_area_harvested`, `Tracks_production`, `Tracks_yield`, `Tracks_price`.

Value

A data.table in which each row corresponds to a crop-year, with standardized track record measures for area planted, area harvested, production, yield, and price.

`get_nass_production_data`

Retrieve and Aggregate NASS Production Data

Description

This helper function cleans, and aggregates production and area data from the USDA NASS Quick Stats API (via your `process_nass_dataset()` function), for specified geographic aggregation levels (national, state, county). It returns a `data.table` summarizing mean production and area by the chosen levels.

Usage

```
get_nass_production_data(
  dir_source = "./data-raw/fastscratch/nass/",
  source_desc = "SURVEY",
  agg_level_desc = c("NATIONAL", "STATE", "COUNTY")
)
```

Arguments

`dir_source`	Character. Path to the directory where NASS Quick Stats raw QS files are stored (default: `"./data-raw/fastscratch/nass/"`).
`source_desc`	Character. The `source_desc` filter passed to NASS (e.g. `"SURVEY"`).
`agg_level_desc`	Character vector. Which aggregation levels to include: any combination of `"NATIONAL"`, `"STATE"`, and `"COUNTY"`. Controls which code columns (`state_code`, `county_code`) are added.

Details

This function begins by constructing the set of grouping keys (`agg_level_list`), always including the year, commodity name, aggregation descriptor, statistic category, and unit, and then conditionally adding state and/or county codes if those levels are requested. It then invokes `process_nass_dataset()` to fetch the raw crop data for the specified source, sector, domain, country, frequency, reference period, statistic categories, and aggregation levels. Once the data are loaded, any invalid or missing values are removed and the mean of the remaining values is computed for each unique combination of metadata columns. Four separate summaries are then generated: (1) overall production/utilization/class totals, (2) breakdown by commodity class, (3) breakdown by utilization practice, and (4) breakdown by production practice. These four summaries are merged back together, and any rows with unwanted units (e.g., containing a dollar sign) or total rows in the commodity name are filtered out. Next, area metrics are processed by selecting the first non-missing sum across the four summaries and averaging it, and production metrics are handled similarly after filtering out invalid unit-commodity combinations and converting cotton bale values to pounds. Finally, the area and production results are bound together and summed across the chosen grouping keys to produce the final `data.table`.

Value

A `data.table` with one row per combination of:

- `commodity_year`, `commodity_name`,
- chosen geographic codes (`state_code`, `county_code`),
- plus any other aggregation keys. Columns contain summed mean values for production and area.

See Also

- `downloaded_nass_large_datasets` - for downloading and caching USDA NASS Quick Stats large dataset files
- `process_nass_dataset` - for retrieving raw NASS Quick Stats data

`get_prism_seasonal_data` *Download and prepare PRISM seasonal weather data for a single year*

Description

All weather variables were first aggregated to the county level as the weighted mean of all the PRISM grids within each county, using the proportion of each PRISM grid related to the respective crop's cover based on cropland data layers (CDLs) from NASS CropScape (2008-2022) as the aggregation weights. Subsequently, the weather variables were further aggregated over each year's growing season for each crop and county.

This function fetches those **pre-aggregated** (county/crop/year) growing-season summaries for a single year, filters to a chosen climate scenario (`SimNum`), standardizes column names, optionally converts precipitation units, and averages across commodity *types* within each `commodity_code` (relevant for wheat).

Usage

```
get_prism_seasonal_data(
  year,
  simnum = 0L,
  convert_precip = c("inch", "mm", "none"),
  cache_dir = NULL,
  overwrite = FALSE,
  quiet = TRUE
)
```

Arguments

`year`	Integer; the year to download (e.g., `2020`).
`simnum`	Integer climate-scenario code to keep (default `0`, observed).
`convert_precip`	One of `"inch"` (default; converts mm to inches), `"mm"` (ensure millimeters), or `"none"` (leave as-is).
`cache_dir`	Optional directory to cache the `.rds` file. Defaults to `tempdir()`.
`overwrite`	Logical; if `TRUE`, re-download even if cached. Default `FALSE`.
`quiet`	Logical; silence `download.file()` output. Default `TRUE`.

Details

Climate scenarios (`SimNum`):

- `SimNum` shows climate scenarios of +0.0, +0.5, +1.0, +1.5, +2.0, +2.5, +3.0 that are derived by adding the respective temperature increase to the observed historic value.
- `SimNum = 0` is the actual observed historic value.

Variable definitions:

- `commodity_year` : commodity year
- `state_code` : state code
- `county_code` : county code
- `commodity_code` : commodity code
  - `ppt` : precipitation (inches if `convert_precip="inch"`, mm if `"mm"`),
  - `tmin` : mean daily minimum temperature (C),
  - `tmax` : mean daily maximum temperature (C),
  - `tavg` : mean daily average temperature (C),
  - `freeze`: exposure to temperatures below 0C down to 100 C,
  - Degree-day summaries `DD*`
    - `DD1` : degree days from 0-10C,
    - `DD2_28` : degree days from 10-28C, ... up to `DD2_35`,
    - `DD3_28` : degree days above 28C, ... up to `DD3_35`.

Value

A `data.table` keyed by `commodity_year, state_code, county_code, commodity_code` with `ppt, tmin, tmax, tavg, freeze` and all `DD*` columns.

Examples

```
## Not run:
library(data.table)
dt <- get_prism_seasonal_data(2018)
dt[]

## End(Not run)
```

---

get_state_rental_rates

Get State Rental Rates for Cropland

Description

Get State Rental Rates for Cropland

Usage

```
get_state_rental_rates(dir_source = "./data-raw/fastscratch/nass/")
```

Arguments

`dir_source` `character(1)` Path to the directory where Quick Stats files are stored. Defaults to `"./data-raw/fastscratch/nass/"`.

Approximate per-acre cost of crop production using state-level rental rates retrieved from USDA NASS Quick Stats. This function:

1. Loads and aggregates NASS asset values and cash rents by state and year.
2. Joins the two series, excludes non-contiguous states/territories.
3. Estimates missing rents via a panel regression on log asset values, then corrects for systematic bias.
4. Interpolates any remaining missing values using a 5-nearest-neighbor spatial average, iterated twice.

Details

Internally this function relies on:

- `process_nass_dataset()` to pull NASS Quick Stats.
- `plm::pdata.frame()` for panel data setup.
- `lm()` to fit a state-fixed-effects trend model.
- `spdep` to compute spatial lags (5-nearest neighbors).
- `doBy::summaryBy()` for error-ratio corrections.
- `terra` and `tigris` to obtain state geometries.

Value

A `data.frame` with columns:

`NAME` State name

`state_code` Numeric state FIPS code

`commodity_year` Year of the observation

`rent` Adjusted per-acre rent ($/acre)

Examples

```
## Not run:
# Make sure `process_nass_dataset()` and required packages are loaded
df <- get_state_rental_rates()
head(df)

## End(Not run)
```

---

harmonize_codes_and_names

Harmonize names/codes in a data table of insurance elections

Description

This function looks for two columns and creates recoded versions grouping similar codes into broader categories.

Usage

```
harmonize_codes_and_names(df)
```

Arguments

df	A `data.frame`

Value

A `data.frame` with the same columns as `df` with new columns

---

harmonize_crop_type_codes

Harmonize and summarize crop type codes from raw SOBTPU data

Description

Download the raw Summary of Business by Type, Practice, and Unit Structure (SOBTPU) data, clean and harmonize the crop type names for a fixed set of commodity codes, determine a single dominant type per county, and save the resulting lookup table of type renames.

Usage

```
harmonize_crop_type_codes()
```

Details

This helper function performs the following steps:

1. Downloads the raw SOBTPU data and reads it in.
2. Filters out rows with missing, zero, infinite, or NaN `liability_amount`, and retains only the specified set of commodity codes (wheat, barley, rice, etc.).
3. Recodes `type_name` by commodity:
   - **Wheat (11):** `"DURUM"`, `"SPRING"`, or `"NO TYPE SPECIFIED"`, dropping `"FORAGE WHEAT FOR SEED"`.
   - **Barley (91):** `"SPRING"` or `"WINTER."`
   - **Rice (18):** `"SHORT"`, `"LONG"`, `"MEDIUM"`, or `"NO TYPE SPECIFIED"`, dropping certain japonica types.
   - **Other crops (41, 75, 94, 17, 81):** `"GRAIN"`, `"SILAGE"` (when `type_code == 26`), or "ALL".
4. Summarizes total `liability_amount` by `(commodity_code, state_code, county_code, type_name)`.
5. Pivots to wide form, with one column per `type_name`, filling missing with zero.
6. Determines the single "dominant" type for each county-i.e. the one with positive liability when all other types are zero.
7. Joins this `type_recode` back onto the full dataset and applies final cleanup:
   - If `type_name` is not `"NO TYPE SPECIFIED"`, it overrides the county-level rename.
   - Fills any remaining blanks or `NA` in `type_recode` with `"NO TYPE SPECIFIED"`.
8. Drops any residual invalid liability rows, selects and deduplicates the key columns, adds a `data_source` column, saves to `./data-raw/type_recodes.rds`, and returns the result.

Value

A `data.frame` with columns:

- `commodity_year`
- `state_code`
- `county_code`
- `commodity_code`
- `type_code`
- `type_recode`
- `data_source` (always "Summary of business by type, practice, and unit structure")

Examples

```
## Not run:
# Build and retrieve the lookup of harmonized crop types
lookup_dt <- harmonize_crop_type_codes()

## End(Not run)
```

layouts_fcip — *Column Layouts for Federal Crop Insurance Program (FCIP) Data*

Description

Use these layouts when reading raw FCIP CSV exports or when constructing data frames for analysis. Ensures consistent column ordering and naming across multiple report types.

Usage

```
layouts_fcip
```

Format

An object of class `list` of length 8.

Source

USDA-RMA

locate_download_link — *Locate the download link for the actuarial data master*

Description

Locate the download link for the actuarial data master

Usage

```
locate_download_link(
  year = 2012,
  adm_url = "https://pubfs-rma.fpac.usda.gov/pub/References/actuarial_data_master/",
  ice_url =
   "https://pubfs-rma.fpac.usda.gov/pub/References/insurance_control_elements/PASS/",
  data_source = "adm"
)
```

Arguments

`year`	the year of the actuarial data master to download
`adm_url`	the url where the ADM FTP site is
`ice_url`	the url where the ICE (insurance control elements) FTP site is
`data_source`	either "adm" or "ice". Defaults to "adm".

Value

a list of the data and layout file urls with the time the file was last updated on RMA's server

Source

copied from https://github.com/dylan-turner25/rmaADM/blob/main/R/helpers.R

Examples

```
## Not run: locate_download_link(year = 2012)
```

---

plot_liability_and_acres

Plot Liability and Net Reported Acres Faceted by a Grouping Variable

---

Description

Produces a two-panel bar chart:

1. **Panel A** - Liability (in U.S.\ dollars) by year and group.
2. **Panel B** - Net reported acres (in acres) by year and group.

The panels share the same x-axis (commodity years) and a single legend that is displayed beneath the figure.

Usage

```
plot_liability_and_acres(
  data,
  grouping_variable,
  grouping_name = NULL,
  time_scale_theme = NULL,
  general_theme = NULL,
  label_liability = "Billion U.S. dollars\n",
  label_net_reported_acres = "Million acres\n",
  palette = c("#003524", "#00583D", "#A0BD78", "#BED73B", "#BE5E27", "#FFC425",
    "#FEF389", "#D7E5C8", "#9DD9F7", "#51ABA0", "#0F374B")
)
```

Arguments

`data` A `data.frame` containing at least these columns:

- `commodity_year` - numeric or factor; the x-axis.
- `value` - numeric; the bar heights.
- `variable` - factor with the levels `"(A) Liability in U.S. dollars"` and `"(B) Net reported acres"`.
- The column referenced by `grouping_variable`.

`grouping_variable` A character scalar giving the name of the column in `data` that defines the groups (e.g.\ `"commodity"`, `"state"`).

`grouping_name` Optional character scalar used as the legend title. Defaults to the value of `grouping_variable` if `NULL`.

`time_scale_theme`
: Optional `ggplot2` scale or theme element that controls the x-axis breaks/labels. If `NULL`, the function applies `scale_x_continuous(breaks = unique(data$commodity_year), labels = unique(data$commodity_year))`.

`general_theme`
: Optional `ggplot2` theme applied to both panels. If `NULL`, `ers_theme()` is used with additional tweaks for font sizes, legend layout, and facet strips.

`label_liability`
: Y-axis label for Panel A. Default `"Billion U.S. dollars\\n"`.

`label_net_reported_acres`
: Y-axis label for Panel B. Default `"Million acres\\n"`.

`palette`
: A character vector of hex colors used to fill the bars. The default is an 11-color palette aligned with ERS/NDSU branding.

Details

Internally the function:

1. Duplicates the column named by `grouping_variable` into `data$grouping_variable` for convenient aesthetic mapping.
2. Builds two separate `ggplot` bar charts (one per `variable` level), applies user themes and palettes, and hides their legends.
3. Extracts a shared legend and x-axis grob with `gtable_filter()`.
4. Assembles everything with `gridExtra::grid.arrange()`.

Value

A named list with five objects:

`fig` A `gtable` containing the assembled two-panel figure, shared x-axis label, and bottom legend.

`figA` The `ggplot` object for Panel A.

`figB` The `ggplot` object for Panel B.

`ldgnd` The extracted legend as a `gtable`.

`xlabT` The shared x-axis grob as a `gtable`.

Examples

```
## Not run:
# Group by commodity with a custom x-axis theme
plot_liability_and_acres(
  data = data_comm,
  grouping_variable = "commodity",
  grouping_name = "Commodity",
  time_scale_theme = ggplot2::scale_x_continuous(
    breaks = seq(2008, 2024, 2),
    labels = seq(2008, 2024, 2)
  ),
  general_theme = ggplot2::theme_minimal()
)

## End(Not run)
```

plot_us_states_choropleth

Plot an Annotated US States Choropleth Map

Description

Creates a choropleth map of US states colored by a categorical value, with state labels and special treatment for small states (VT, NH, and other small ones) using repelled text. Optionally embeds a table grob in the bottom-left corner of the map.

Usage

```
plot_us_states_choropleth(
  data,
  legend_title = NULL,
  palette = c("#003524", "#00583D", "#A0BD78", "#BED73B", "#BE5E27", "#FFC425",
    "#FEF389", "#D7E5C8", "#9DD9F7", "#51ABA0", "#0F374B"),
  table_grob = NULL,
  label_size = 2.5
)
```

Arguments

`data`	A data frame containing at least: • `state_code`: numeric FIPS code matching `urbnmapr::get_urbn_map("states")` • `value_cat`: categorical variable used for fill colors • `value`: numeric variable used to display numeric labels
`legend_title`	Character; title for the fill legend. If `NULL`, no title is shown.
`palette`	Character vector of colours (hex codes) to use for the categories. Defaults to a 10-colour NDSU-inspired palette.
`table_grob`	A `grob` object (e.g. from `gridExtra::tableGrob`) to annotate on the map; if `NULL`, no table is added.
`label_size`	label size for sgeom_sf_text

Details

- Uses `urbnmapr::get_urbn_map(map = "states", sf = TRUE)` to fetch a US states basemap.
- Joins the input data on `state_code` and filters out states with missing `value_cat`.
- Computes equal-area centroids (EPSG:5070) to place labels.
- Flags states with area < 50,000 km^2 as "small" and applies repelled text labels.
- Standard states get text labels placed via `geom_sf_text()`.
- Small states in the east and west are nudged horizontally; VT & NH get custom nudges.
- Adds an optional table in the bottom-left via `annotation_custom()`.

Value

A `ggplot` object showing the US states choropleth with annotated labels.

Examples

```
## Not run:
library(sf)
# Assume `my_data` has columns: state_code, value_cat, value
gg <- plot_us_states_choropleth(
  data = my_data,
  legend_title = "Category",
  palette = c("#003524", "#00583D", "#A0BD78", "#BED73B",
              "#BE5E27", "#FFC425", "#FEF389", "#D7E5C8",
              "#9DD9F7", "#51ABA0", "#0F374B")
)
print(gg)

## End(Not run)
```

---

policytime — *Add U.S. Farm Policy Vertical Lines and Labels to a ggplot*

Description

This function overlays vertical lines and text labels on a ggplot object to mark major U.S. agricultural policy events (e.g., Farm Bills or Acts). The vertical lines are drawn at specific years, and labels are positioned based on values provided in the `pty` vector.

Usage

```
policytime(pty, plot, size)
```

Arguments

`pty`	A numeric vector of y-axis positions for label placement. The length of the vector determines which policy annotations are added: • 1st: 1980 Act • 2nd: 1994 Act • 3rd: 1996 Farm Bill • 4th: 2000 Agricultural Risk Protection Act • 5th: 2008 Farm Bill • 6th: 2014 Farm Bill • 7th: 2018 Farm Bill
`plot`	A `ggplot` object to which policy lines and labels will be added.
`size`	A numeric value indicating the text size of the policy labels.

Details

The lines and labels are added in brown color with dashed lines (`lty=5`), and labels are rotated vertically. Labels are only drawn if the corresponding index exists in `pty`.

Value

A `ggplot` object with added vertical dashed lines and corresponding text labels for each policy year provided.

Examples

```
## Not run:
base_plot <- ggplot(data, aes(x = commodity_year, y = value)) +
  geom_line()
policy_positions <- c(10, 15, 20, 25, 30, 35, 40)
annotated_plot <- policytime(policy_positions, base_plot, size = 3)
print(annotated_plot)

## End(Not run)
```

`process_nass_dataset` *Process a USDA NASS Quick Stats dataset by sector and statistic category*

Description

`process_nass_dataset()` downloads (if needed) and reads one or more NASS Quick Stats large datasets files for a given sector, filters the rows by the chosen statistic category plus any additional Quick Stats API parameters, converts and cleans the `value` column, aggregates it by taking its mean over all remaining grouping columns, and then renames that aggregated column to match the requested statistic.

Usage

```
process_nass_dataset(
  dir_source = "./data-raw/fastscratch/nass/",
  large_dataset,
  statisticcat_desc = NULL,
  nassqs_params = NULL
)
```

Arguments

`dir_source` `character(1)` **Length 1.** Path to the directory where Quick Stats large datasets files are stored (and will be downloaded to via `get_nass_large_datasets()`). Defaults to `"./data-raw/fastscratch/nass/"`.

`large_dataset` `character(1)` The Quick Stats `large_dataset` to load (e.g. `"crops"`). one of: "census2002","census2007","census2012","census2017","census2022", "census2007zipcode","censu "animals_products","crops","demographics","economics","environmental"

`statisticcat_desc` `character(1)` **Length 1.** The Quick Stats `statisticcat_desc` to filter on (e.g. `"PRICE RECEIVED"`). After aggregation, the resulting column of mean values will be renamed to `gsub(" ", "_", statisticcat_desc)`.

`nassqs_params` `list` or `NULL` A named list of additional Quick Stats API parameters to filter by (e.g. `"domain_desc"`, `"agg_level_desc"`, `"year"`, etc.). Names must correspond to valid Quick Stats fields. If `NULL` (the default), only `sector_desc` + `statisticcat_desc` filtering is applied. Use `rnassqs::nassqs_params()` to list all valid parameter names.

Details

The full set of valid Quick Stats API parameter names can be retrieved with:

```
rnassqs::nassqs_params()
```

Value

A `data.table` where:

- All original columns have been lowercased.
- Rows have been filtered by `nassqs_params`.
- A `value` column has been converted to numeric (commas stripped), cleaned of non-finite entries, and then aggregated by mean over the remaining columns.
- That aggregated column is renamed to `gsub(" ", "_", statisticcat_desc)`.
- Numeric code columns `state_code`, `country_code`, `asd_code`, plus `commodity_year` and `commodity_name` have been created.

See Also

- `get_nass_large_datasets()` for downloading the raw Quick Stats files

Examples

```
## Not run:
# National annual average price received for all CROPS in 2020:
dt1 <- process_nass_dataset(
  large_dataset       = "crops",
  statisticcat_desc = "PRICE RECEIVED",
  nassqs_params = list( agg_level_desc = "NATIONAL", year = 2020 ))

# State-level marketing-year average price for soybeans:
dt2 <- process_nass_dataset(
  large_dataset       = "crops",
  statisticcat_desc = "PRICE RECEIVED",
  nassqs_params     = list(
    agg_level_desc      = "STATE",
    short_desc          = "SOYBEANS - PRICE RECEIVED, MEASURED IN $ / BU",
    reference_period_desc = "MARKETING YEAR",
    freq_desc           = "ANNUAL"
  )
)

## End(Not run)
```

---

push_to_overleaf *Push R Markdown Output to Overleaf via Git*

Description

This is the main function that renders an R Markdown file and pushes the result (and any additional files) to an Overleaf Git project.

Usage

```
push_to_overleaf(
  rmd_file,
  overleaf_project_id,
  local_dir = "overleaf_project",
  figure_files = NULL,
  commit_msg = "Update from R",
  auth_token = Sys.getenv("OVERLEAF_TOKEN"),
  git_user_name = NULL,
  git_user_email = NULL
)
```

Arguments

`rmd_file`	Path to the R Markdown (.Rmd) file.
`overleaf_project_id`	Overleaf Git project ID (from Overleaf Git settings).
`local_dir`	Local directory where the repo will be cloned (default: `"overleaf_project"`).
`figure_files`	Optional vector of file paths (e.g., images, .bib) to copy to the Overleaf repo.
`commit_msg`	Git commit message (default: `"Update from R"`).
`auth_token`	Overleaf Git token. Defaults to `Sys.getenv("OVERLEAF_TOKEN")`.
`git_user_name`	Git username to configure locally in the repo (optional).
`git_user_email`	Git email to configure locally in the repo (optional).

Value

Invisibly returns `TRUE` if successful, `FALSE` otherwise.

`render_rmarkdown_to_dir` *Render R Markdown to a Target Directory with Environment Backup*

Description

Safely renders an R Markdown document to a specified output directory. Temporarily saves and restores the global environment in case the Rmd contains destructive commands like `rm(list = ls())`.

Usage

```
render_rmarkdown_to_dir(rmd_file, output_dir)
```

Arguments

`rmd_file`	Path to the `.Rmd` file to render.
`output_dir`	Directory to render output files into.

Value

Invisibly returns `TRUE` on success. Will `stop()` if rendering fails.

rescale_values — *Rescale Numeric Values for Readable Display*

Description

This function rescales each numeric value in a vector independently to ensure at most three significant digits appear before the decimal point. It then returns a character vector of rounded values with the appropriate unit label.

Usage

```
rescale_values(x, rounding = 2)
```

Arguments

x	A named numeric vector to rescale.
rounding	Number of decimal places to round the scaled value (default is 2).

Value

A character vector of formatted values with their scale unit (e.g., "156.54 million").

revealed_aip_state_operation — *Build a state-level* **revealed** *Plan of Operations from historical SRA data*

Description

Pulls public SRA history from `rfcip` (both `nationalSRA` and `stateSRA`), normalizes fund labels, collapses to one row per `state-commodity_year-fund`, and returns a wide, state-year table with (i) each fund's share of the state book and (ii) the average premium-retention percentage retained inside each fund.

Usage

```
revealed_aip_state_operation()
```

Details

The result is a ready-made `aip_state_operation`-style table that can be used by simulation helpers such as `fund_alocation_and_retention()`.

Steps performed:

1. Bind the national aggregate (tagged as `state == "US"`) to state-level SRA records.
2. Drop roll-up rows (`fund_abb == "Total"`).
3. Map FCIC fund abbreviations to `arf`, `cf`, `df`: `CC/OC/RC/C -> cf`, `CD/OD/RD -> df`, `AR/OA/A -> arf`.
4. Keep premium fields and set `commodity_year <- reinsurance_year`.

5. Sum dollars by `state` × `commodity_year` × `fund_abb` × `value_type`.
6. Reshape wide by `value_type` to get `gross_premium` and `retained_premium`; compute:

$$share = \frac{\text{gross_premium}}{\sum_{\text{fund}} \text{gross_premium}}$$

$$retention = \frac{\text{retained_premium}}{\text{gross_premium}}$$

7. Average across multiple company records within the same state-fund-year.
8. Reshape to a wide, state-year layout with fund-prefixed columns.

Notes:

- Missing values are handled with `na.rm = TRUE` in sums/means.
- Keys in the final wide table are `state` and `commodity_year`.
- The output includes **shares** and **retention** plus additional fund-prefixed aggregates (e.g., gross/retained premium and indemnity).

Value

A `data.table` with one row per `state-commodity_year` and columns:

- `state` - two-letter postal code; `"US"` for the national aggregate.
- `commodity_year` - FCIC reinsurance year (treated as commodity year).
- Fund-share and retention columns: `arf_share`, `cf_share`, `df_share`; `arf_retention`, `cf_retention`, `df_retention`.
- Additional fund-prefixed metrics (averaged over companies where applicable): `<fund>_gross_indemnity`, `<fund>_gross_liability`, `<fund>_gross_premium`, `<fund>_net_gain_loss`, `<fund>_retained_indemnity`, `<fund>_retained_liability`, `<fund>_retained_premium`, where `<fund> %in% c("arf","cf","df")`.

Dependencies

Requires **rfcip** for `nationalSRA` and `stateSRA`; uses **data.table**, **dplyr**, and **tidyr** for aggregation and reshaping.

Examples

```
## Not run:
dt <- revealed_aip_state_operation()
dt[state == "IA" & commodity_year >= 2018,
   .(IA_arf_share = arf_share, IA_cf_share = cf_share, IA_df_share = df_share)]

## End(Not run)
```

setup_environment *Setup Project Environment*

Description

Initializes the working environment for a project by creating required directories, setting useful global options, and fixing the random seed.

Usage

```
setup_environment(
  year_beg = 2001,
  year_end = as.numeric(format(Sys.Date(), "%Y")),
  seed = 1980632,
  project_name,
 local_directories = list(file.path("data-raw", "output"), file.path("data-raw",
    "scripts"), file.path("data")),
  fastscratch_root = NULL,
  fastscratch_directories = NULL
)
```

Arguments

`year_beg` Integer. Beginning year of the analysis (default: 2001).

`year_end` Integer. Ending year of the analysis (default: current system year).

`seed` Integer. Random seed for reproducibility (default: 1980632).

`project_name` Character. Project name (required). Used to build fast-scratch directory paths.

`local_directories` List of project-local directories to create (default: `list("data-raw/output", "data-raw/scripts", "data")`).

`fastscratch_root` Optional character. Root directory for fast-scratch files. If `NULL`, it is set automatically:

- Windows: `"C:/fastscratch"`
- Linux/macOS: `"/fastscratch/<username>"`

`fastscratch_directories` List of fast-scratch subdirectories (relative to `<fastscratch_root>/<project_name>`) to create. If `NULL`, no fast-scratch subdirectories are created and `wd` is returned as an empty list.

Details

The function ensures the requested directories exist, creating them if necessary. Directory keys in the returned `wd` list are the basenames of the provided `fastscratch_directories`.

It also sets the following options:

- `options(scipen = 999)` (turns off scientific notation)
- `options(future.globals.maxSize = 8 * 1024^3)` (~8 GiB)
- `options(dplyr.summarise.inform = FALSE)` (quiet **dplyr**)

Finally, the random number generator is seeded with the provided `seed`.

Value

A list with:

wd Named list of created fast-scratch directories. Empty if `fastscratch_directories = NULL`.

year_beg Starting year (integer).

year_end Ending year (integer).

seed Seed value used for RNG.

Examples

```
## Not run:
env <- setup_environment(
  year_beg = 2015,
  year_end = 2024,
  project_name = "HiddenSafetynet2025",
  fastscratch_directories = c("output/sims", "output/expected")
)
str(env$wd)

## End(Not run)
```

---

setup_git_repo — *Clone and Configure Overleaf Git Repository*

Description

Clones the Overleaf Git repository if it doesn't already exist locally, and sets local Git user identity if provided.

Usage

```
setup_git_repo(
  overleaf_git_url,
  local_dir,
  user_name = NULL,
  user_email = NULL
)
```

Arguments

`overleaf_git_url`	Full Git clone URL with embedded token.
`local_dir`	Directory to clone the repo into (if not already cloned).
`user_name`	Git username (optional). If provided, set locally for the repo.
`user_email`	Git email (optional). If provided, set locally for the repo.

Value

Invisibly returns `TRUE`. Will `stop()` on clone failure.

smooth_county_estimates

Smooth county-level estimates via spatial contiguity

Description

This function

1. Loads all U.S. county geometries via **urbnmapr**
2. Joins in a user-supplied data.frame of county estimates (by 5-digit FIPS)
3. (Optionally) converts to a **terra** SpatVector for further terra ops
4. Builds a contiguity neighbor list using **sf::st_touches**
5. Iteratively fills any missing estimates by applying a user-supplied function to neighbor values
6. Classifies the smoothed values into `n_classes` using Jenks natural breaks

Usage

```
smooth_county_estimates(
  data,
  fip_col = "fip",
  estimate_col = "estimate",
  iterations = 5,
  fun = mean,
  n_classes = 10
)
```

Arguments

`data`	A data.frame or tibble containing at least two columns: - a 5-digit FIPS code (character) - a numeric estimate to be smoothed
`fip_col`	Name of the FIPS-code column in `fit_df` (default 'fip').
`estimate_col`	Name of the numeric estimate column in `fit_df` (default 'estimate').
`iterations`	Number of smoothing passes to perform (default 5).
`fun`	A function to aggregate neighbor values; must take a numeric vector and return a single numeric (default mean).
`n_classes`	Number of categories for Jenks natural-breaks classification (default 10).

Value

An **sf** object of all U.S. counties with these new columns:

- estimate: your original values (NA where missing)
- estimate_smooth: the same values after neighborhood-function imputation
- estimate_cat: factor giving the Jenks-break category

standardize_fcip_column_names

Standardize FCIP Data Frame Column Names

Description

Rename a set of common Federal Crop Insurance Program (FCIP) data frame columns to consistent, descriptive names. This ensures uniform naming across different data sources and simplifies downstream processing.

Usage

```
standardize_fcip_column_names(df)
```

Arguments

df	A data.frame containing FCIP data with short column names such as crop_yr, state_cd, etc.

Value

The input df with Standardized column names:

Examples

```
## Not run:
# Suppose fcip_raw is loaded with original column names
fcip_clean <- standardize_fcip_column_names(fcip_raw)
# Now fcip_clean has standardized, descriptive column names

## End(Not run)
```

Index

∗ **FCIP Re-insurance Calculators**
revealed_aip_state_operation, 47
∗ **datasets**
FCIP_FORCE_AMOUNT_VARIABLES, 20
FCIP_FORCE_CHARACTER_KEYS, 20
FCIP_FORCE_NUMERIC_KEYS, 21
FCIP_INSURANCE_ELECTION, 22
FCIP_INSURANCE_ELECTION_RCODED, 22
FCIP_INSURANCE_POOL, 23
layouts_fcip, 39

analyze_and_summarize_time_series, 3

calculate_mode, 4
calculate_revenue_moments, 5
clean_data, 6
clean_eco_share_by_insurance_plan, 6
clean_rma_sobtpu, 7
clean_sco_share_by_coverage_level, 8
clean_supplemental_plan_shares, 8
commit_and_push, 9

download_rma_web_data_files, 11
downloaded_nass_large_datasets, 10, *34*

enrich_sobtpu_with_covariates, 12
enrich_with_commodity_grouping, 14
enrich_with_election_grouping, 14
enrich_with_location_grouping, 15
enrich_with_loss_cause, 15
enrich_with_loss_ratio_grouping, 16
enrich_with_practice_grouping, 17
enrich_with_program_age, 17
enrich_with_structural_breaks, 18
enrich_with_subsidy_bins, 19
ers_theme, 19

FCIP_FORCE_AMOUNT_VARIABLES, 20
FCIP_FORCE_CHARACTER_KEYS, 20
FCIP_FORCE_NUMERIC_KEYS, 21
FCIP_INSURANCE_ELECTION, 22
FCIP_INSURANCE_ELECTION_RCODED, 22
FCIP_INSURANCE_POOL, 23

generate_toy_farmer_data, 24
get_continuous_rating_parameters, 26
get_file_info, 27
get_ice_data, 28
get_marketing_year_avg_price, 29
get_nass_census_data, 30
get_nass_historical_track_record_crop, 32
get_nass_production_data, 33
get_prism_seasonal_data, 34
get_state_rental_rates, 36

harmonize_codes_and_names, 37
harmonize_crop_type_codes, 37

layouts_fcip, 39
locate_download_link, 39

plot_liability_and_acres, 40
plot_us_states_choropleth, 42
policytime, 43
process_nass_dataset, *31*, *34*, 44
push_to_overleaf, 45

render_rmarkdown_to_dir, 46
rescale_values, 47
revealed_aip_state_operation, 47
rfcip::get_sob_data(), *7*

setup_environment, 49
setup_git_repo, 50
smooth_county_estimates, 51
standardize_fcip_column_names, 52